

Symptom Description Predictor Project

Complete Data Science Project Pipeline using Google Colab

```
In [ ]: from google.colab import files
        uploaded = files.upload()
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving symptom_Description.csv to symptom_Description.csv

```
In [ ]: # 2. Load the Dataset
        import pandas as pd
        df = pd.read_csv('symptom_Description.csv')
        df.head()
```

```
Out[ ]:
```

	Disease	Description
0	Drug Reaction	An adverse drug reaction (ADR) is an injury ca...
1	Malaria	An infectious disease caused by protozoan para...
2	Allergy	An allergy is an immune system response to a f...
3	Hypothyroidism	Hypothyroidism, also called underactive thynoi...
4	Psoriasis	Psoriasis is a common skin disorder that forms...

```
In [ ]: # 3. Data Exploration
        df.info()
        df.describe(include='all')
        df.shape
        df.columns

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41 entries, 0 to 40
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   Disease         41 non-null    object 
 1   Description     41 non-null    object 
dtypes: object(2)
memory usage: 788.0+ bytes
```

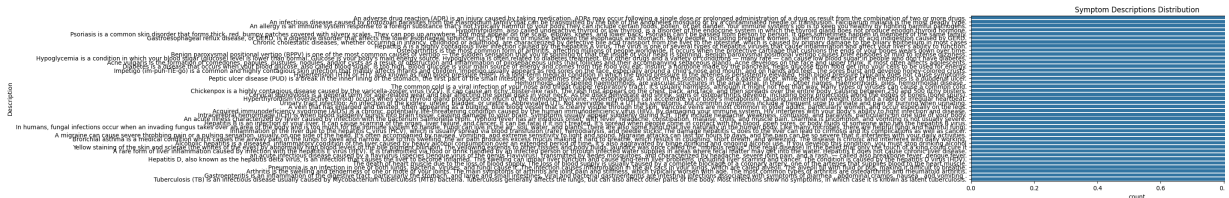
```
Out[ ]: Index(['Disease', 'Description'], dtype='object')
```

```
In [ ]: # 4. Check for Missing Values and Duplicates
        print(df.isnull().sum())
        print("Duplicates:", df.duplicated().sum())

Disease      0
Description  0
dtype: int64
Duplicates: 0
```

```
In [ ]: # 5. Visualize a Few Features
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,5))
sns.countplot(y='Description', data=df)
plt.title('Symptom Descriptions Distribution')
plt.show()
```



```
In [ ]: # 6. Identify Target and Features
# Example: Replace 'Disease' with your actual target column name
X = df['Description']
# y = df['Disease'] # Uncomment and replace if target exists
```

```
In [ ]: # 7. Convert Categorical Columns to Numerical
# y = y.astype('category').cat.codes # Uncomment if applicable
```

```
In [ ]: # 8. One-Hot Encoding
X = pd.get_dummies(X)
```

```
In [ ]: # 9. Feature Scaling
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
In [ ]: # 10. Train-Test Split
from sklearn.model_selection import train_test_split

# Replace y with actual target
# X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_s:
```

```
In [ ]: # 11. Model Building
from sklearn.ensemble import RandomForestClassifier

# model = RandomForestClassifier()
# model.fit(X_train, y_train)
```

```
In [ ]: # 12. Evaluation
from sklearn.metrics import accuracy_score, classification_report

# y_pred = model.predict(X_test)
# print("Accuracy:", accuracy_score(y_test, y_pred))
# print(classification_report(y_test, y_pred))
```

```
In [ ]: # 13. Make Predictions from New Input
new_input = ['sore throat']
```

```
In [ ]: # 14. Convert to DataFrame and Encode
        new_df = pd.DataFrame(new_input, columns=['Description'])
        new_df_encoded = pd.get_dummies(new_df)
        new_df_encoded = new_df_encoded.reindex(columns=X.columns, fill_value=0)
        new_scaled = scaler.transform(new_df_encoded)

In [ ]: # 15. Predict the Final Grade
        # prediction = model.predict(new_scaled)
        # print("Predicted class:", prediction[0])

In [ ]: # 16. Deployment - Building an Interactive App
        !pip install gradio
        import gradio as gr
```

Collecting gradio
 Downloading gradio-5.29.0-py3-none-any.whl.metadata (16 kB)
 Collecting aiofiles<25.0,>=22.0 (from gradio)
 Downloading aiofiles-24.1.0-py3-none-any.whl.metadata (10 kB)
 Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (4.9.0)
 Collecting fastapi<1.0,>=0.115.2 (from gradio)
 Downloading fastapi-0.115.12-py3-none-any.whl.metadata (27 kB)
 Collecting ffmpeg (from gradio)
 Downloading ffmpeg-0.5.0-py3-none-any.whl.metadata (3.0 kB)
 Collecting gradio-client==1.10.0 (from gradio)
 Downloading gradio_client-1.10.0-py3-none-any.whl.metadata (7.1 kB)
 Collecting groovy~=0.1 (from gradio)
 Downloading groovy-0.1.2-py3-none-any.whl.metadata (6.1 kB)
 Requirement already satisfied: httpx>=0.24.1 in /usr/local/lib/python3.11/dist-packages (from gradio) (0.28.1)
 Requirement already satisfied: huggingface-hub>=0.28.1 in /usr/local/lib/python3.11/dist-packages (from gradio) (0.30.2)
 Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (3.1.6)
 Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (3.0.2)
 Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (2.0.2)
 Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (3.10.18)
 Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from gradio) (24.2)
 Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (2.2.2)
 Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (11.2.1)
 Requirement already satisfied: pydantic<2.12,>=2.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (2.11.4)
 Collecting pydub (from gradio)
 Downloading pydub-0.25.1-py2.py3-none-any.whl.metadata (1.4 kB)
 Collecting python-multipart>=0.0.18 (from gradio)
 Downloading python_multipart-0.0.20-py3-none-any.whl.metadata (1.8 kB)
 Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (6.0.2)
 Collecting ruff>=0.9.3 (from gradio)
 Downloading ruff-0.11.8-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (25 kB)
 Collecting safehttpx<0.2.0,>=0.1.6 (from gradio)
 Downloading safehttpx-0.1.6-py3-none-any.whl.metadata (4.2 kB)
 Collecting semantic-version~=2.0 (from gradio)
 Downloading semantic_version-2.10.0-py2.py3-none-any.whl.metadata (9.7 kB)
 Collecting starlette<1.0,>=0.40.0 (from gradio)
 Downloading starlette-0.46.2-py3-none-any.whl.metadata (6.2 kB)
 Collecting tomlkit<0.14.0,>=0.12.0 (from gradio)
 Downloading tomlkit-0.13.2-py3-none-any.whl.metadata (2.7 kB)
 Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.11/dist-packages (from gradio) (0.15.3)
 Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (4.13.2)
 Collecting uvicorn>=0.14.0 (from gradio)
 Downloading uvicorn-0.34.2-py3-none-any.whl.metadata (6.5 kB)
 Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from gradio-client==1.10.0->gradio) (2025.3.2)
 Requirement already satisfied: websockets<16.0,>=10.0 in /usr/local/lib/python3.11/dist-packages (from gradio-client==1.10.0->gradio) (15.0.1)

```
In [ ]: # 17. Create a Prediction Function
def predict_symptom(symptom):
    df_input = pd.DataFrame([symptom], columns=['Description'])
    df_input = pd.get_dummies(df_input)
    df_input = df_input.reindex(columns=X.columns, fill_value=0)
    df_scaled = scaler.transform(df_input)
    pred = model.predict(df_scaled)
    return f"Predicted: {pred[0]}"

In [ ]: # 18. Create the Gradio Interface
interface = gr.Interface(fn=predict_symptom,
                        inputs=gr.Textbox(label="Enter Symptom"),
                        outputs="text")

interface.launch()
```

It looks like you are running Gradio on a hosted Jupyter notebook. For the app to work, sharing must be enabled. Automatically setting `share=True` (you turn this off by setting `share=False` in `launch()` explicitly).

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://03a7e838ce32935451.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrade `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (<https://huggingface.co/spaces>)

Out[]:

19. Student Performance Predictor

Your interactive app is now ready to predict symptoms from user input.